

INSTITUTO FEDERAL
Sul-rio-grandense

Câmpus
Pelotas



Rotas Avançadas e REST API no Laravel

Rotas, Recursos, Injeção de Dependência
com Model Binding e Clientes HTTP para Rest API

Prof. Gill Velleda Gonzales
gillgonzales@ifsul.edu.br | @gillgonzales | @g1ll
CSTSI - Novembro - 2024

Rotas Avançadas e Rest API

Tópicos da Aula



- **Agrupamento de Rotas**
 - Por Prefixo, Controladores e Middlewares
- **Recursos e Route Model Binding**
 - Recursos web e Recursos de api
- **Configurando uma API Rest**
 - Instalação do módulo de API do Laravel
 - Rotas com Resource API
 - Configuração de Parâmetros
- **Consumindo com o Cliente HTTP**
 - Configuração de Coleções
 - Usando variáveis de ambiente

Agrupamento de Rotas

Configurações Avançadas de Rotas



- Rotas podem serem agrupadas utilizando o método **group()** do objeto **Route**
- O método **group** deve receber uma **função anônima**, ou Closure no php.
- Os agrupamentos podem acontecer de várias formas, por **prefixo**, **controladores** ou **middlewares**.
- Exemplo de agrupamento por **prefixos**:

routes >  web.php >  Closure

```
11
12 Route::prefix('/produtos')->group(function () {
13     Route::get('/', [ProdutoController::class, 'index'])->name("produto.index");
14     Route::get('/{id}', [ProdutoController::class, 'show'])->name("produto.show");
15 });
16
17 Route::prefix('/produto')->group(function () {
18     Route::get('/', [ProdutoController::class, 'create'])->name("produto.create");
19     Route::post('/', [ProdutoController::class, 'store'])->name("produto.store");
20     Route::get('/{id}/edit', [ProdutoController::class, 'edit'])->name("produto.edit");
21     Route::post('/{id}/update', [ProdutoController::class, 'update'])->name("produto.update");
22 });
```

Agrupamento de Rotas

Configurações Avançadas de Rotas



- Exemplo de agrupamento por **Controladores**:
- Não há necessidade de informar o Controlador a cada nova rota.
- Neste caso passamos a uma única vez a referência da **Classe**, ou seja, com o operador de escopo estático **::** seguido de **class**.

routes >  web.php >  Closure

```
32
33 Route::controller(ProdutoController::class)->group(function(){
34     Route::get('/produtos','index')->name("produto.index");
35     Route::get('/produtos/{produto}','show')->name("produto.show");
36     Route::get('/produto','create')->name("produto.create");
37     Route::post('/produto','store')->name("produto.store");
38     Route::get('/produto/{id}/edit','edit')->name("produto.edit");
39     Route::post('/produto/{id}/update','update')->name("produto.update");
40 });
41
```


Agrupamento de Rotas

Configurações Avançadas de Rotas



- **Combinação de Vários Agrupamentos:**
 - Exemplo de agrupamento por **Controladores e Prefixos**:

```
... routes >  web.php >  Closure
```

```
11 Route::controller(ProdutoController::class)->group(function(){
12     Route::prefix('/produtos')->group(function () {
13         Route::get('/', 'index')->name("produto.index");
14         Route::get('/{id}', 'show')->name("produto.show");
15     });
16
17     Route::prefix('/produto')->group(function () {
18         Route::get('/', 'create')->name("produto.create");
19         Route::post('/', 'store')->name("produto.store");
20         Route::get('/{id}/edit', 'edit')->name("produto.edit");
21         Route::post('/{id}/update', 'update')->name("produto.update");
22     });
23 });
```

Agrupamento de Rotas

Configurações Avançadas de Rotas



- Exemplo de agrupamento por **Middlewares**:
- **Middlewares** são classes intermediárias entre a requisição e o nosso controlador.
- Podemos gerar rotas que usem um mesmo **middleware** usando o **group()**.

a03 > routes >  web.php >  Closure >  Closure

```
7 Route::controller(ProdutoController::class)->group(function(){
3 //Toda manipulação de produto deverá passar pela autenticação
4 Route::middleware('auth')->group(function () {
5     Route::prefix('/produto')->group(function () {
6         Route::get('/', 'create')->name("produto.create");
7         Route::post('/', 'store')->name("produto.store");
8         Route::get('/{id}/edit', 'edit')->name("produto.edit");
9         Route::post('/{id}/update', 'update')->name("produto.update");
0     });
1 });
2 });
```

Recursos e Route Model Bindings

Criação de Recursos



- Recursos são identificados no **Laravel** como **controladores** especializados em **CRUD**.
- Ou seja, um controlador com todos os métodos necessários para as operações de criação (**C**), leitura (**R**), atualização (**U**) e remoção (**D**).
- Podemos criar um controlador do tipo recurso (**resource**), simplesmente trocando o parâmetro **-c** por **-r** na criação de uma **model**.
- Exemplo:

```
$> php artisan make:model Fornecedor -mr
```

```
Message (Ctrl+Enter to ...)
```

✓ Commit | ▾

Changes 3

- php FornecedorController.php U
- php Fornecedor.php app/... U
- php 2024_11_07_141712_... U

```
> php artisan make:model Fornecedor -mr
```

```
INFO Model [app/Models/Fornecedor.php] created successfully.
```

```
INFO Migration [database/migrations/2024_11_07_141712_create_fornecedors_table.php] created successfully.
```

```
INFO Controller [app/Http/Controllers/FornecedorController.php] created successfully.
```


Recursos e Route Model Bindings

Route Model Binding



- Observe que a noca classe **FornecedorController** já possui os métodos necessários para as operações de **CRUD**.
- A assinatura dos métodos **show**, **edit**, e **update** recebem como argumentos um objeto do tipo da **model específica** deste controlador, a **Fornecedor**.
- Portanto, ao passar o **ID do fornecedor** na rota do **show**, este método já receberá o objeto da model pronta em **\$fornecedor** (**Route Model Binding**).

PROTOTIPO-20242-LARAVEL-BACKEND-API

OUTLINE

✓ PHP Intelephense

{ } App\Http\Controllers

✓ FornecedorController

index

create

> store

> show

> edit

> update

> destroy

app > Http > Controllers > FornecedorController.php > PHP Intelephense > FornecedorController

```
8      class FornecedorController extends Controller
15 >      /** ...
18      public function create(){}
19
20 >      /** ...
23      public function store(Request $request){}
24
25 >      /** ...
28      [ public function show(Fornecedor $fornecedor){} ]
29
30 >      /** ...
33      public function edit(Fornecedor $fornecedor){}
```


Recursos e Route Model Bindings

Route Model Binding



- Para configurar as rotas para os métodos do recurso **FornecedorController**, basta usar o método **resource()** da classe **Route**, e as rotas de CRUD serão geradas automaticamente.

```
$> php artisan route:list --path=fornecedor
```

```
routes > web.php > ...

25
26 Route::resource('fornecedores', FornecedorController::class);
27
```

OUTPUT TERMINAL PORTS DEVDB CODE REFERENCE LOG COMMENTS PROBLEMS DEBUG CONSOLE

/media/P/P/i/c/daoo/202402/daoo/prototipos/prototipo-20242-laravel-backend-api main !1

```
> php artisan route:list --path=fornecedor
```

GET HEAD	fornecedores	fornecedores.index > FornecedorController@index
POST	fornecedores	fornecedores.store > FornecedorController@store
GET HEAD	fornecedores/create	fornecedores.create > FornecedorController@create
GET HEAD	fornecedores/{fornecedor}	fornecedores.show > FornecedorController@show
PUT PATCH	fornecedores/{fornecedor}	fornecedores.update > FornecedorController@update
DELETE	fornecedores/{fornecedor}	fornecedores.destroy > FornecedorController@destroy
GET HEAD	fornecedores/{fornecedor}/edit	fornecedores.edit > FornecedorController@edit

Showing [7] routes

IFSUL | CSTSI | Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br | @gillgonzales | @g1ll

Recursos e Route Model Bindings

Route Model Binding



- Para corrigir a **convenção de nomes** do **Laravel** para o parâmetro **fornecedor**, usaremos o método **parameters** e passar a configuração correta.

```
$> php artisan route:list --path=fornecedor
```

```
routes > php web.php > ...  
26 Route::resource('fornecedores', FornecedorController::class)->parameters(  
27     ["fornecedores"=>'fornecedor']  
28 );
```

OUTPUT TERMINAL PORTS DEVDB CODE REFERENCE LOG COMMENTS PROBLEMS DEBUG CONSOLE

zsh + - [] [x]

/media/P/P/i/c/daoo/202402/daoo/prototipos/prototipo-20242-laravel-backend-api main !1

```
> php artisan route:list --path=fornecedor
```

```
GET|HEAD   fornecedores ..... fornecedores.index > FornecedorController@index  
POST       fornecedores ..... fornecedores.store > FornecedorController@store  
GET|HEAD   fornecedores/create ..... fornecedores.create > FornecedorController@create  
GET|HEAD   fornecedores/{fornecedor} ..... fornecedores.show > FornecedorController@show  
PUT|PATCH fornecedores/{fornecedor} ..... fornecedores.update > FornecedorController@update  
DELETE     fornecedores/{fornecedor} ..... fornecedores.destroy > FornecedorController@destroy  
GET|HEAD   fornecedores/{fornecedor}/edit ..... fornecedores.edit > FornecedorController@edit
```

Recursos e Route Model Bindings

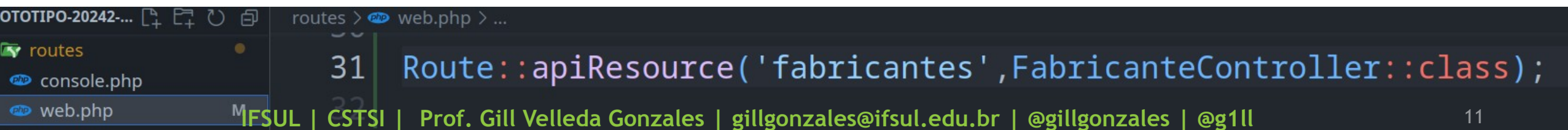
Recursos de API



- **Recurso de API**
- Até o momento vimos a criação de um recurso **web**, para criar um **recurso de API**, ou seja, sem a necessidade dos métodos de geração de *templates* com formulários (**create** e **edit**), usaremos as opções **--resource --api**, no comando:

```
$> php artisan make:model Fabricante -mr --resource --api
```

- Desta forma será criado um controlador de **FabricanteController** na pasta **app/Http/Controllers/**, mas **sem os métodos** voltados para uma **interface web**.
 - Serão criados apenas os métodos **index**, **show**, **store**, **update** e **delete**.
- Já as rotas para este controlador, são geradas com o método **apiResource** da classe **Route**.



Recursos e Route Model Bindings

Recursos de API



• Recurso de API

PROTOTIPO-20242-LARAVEL-BACKEN...

app
Http/Controllers
Controller.php
FabricanteControll... U
FornecedorController.php
ProdutoController.php

OUTLINE

PHP Intelephense
PHP IntelliSense
App\Http\Controllers
FabricanteController
index
store
show
update
destroy

app > Http > Controllers > FabricanteController.php > ...

```
5 use App\Http\Controllers\Controller;  
6 use Illuminate\Http\Request;  
7  
8 class FabricanteController extends Controller
```

OUTPUT TERMINAL PORTS DEVDB CODE REFERENCE LOG COMMENTS PROBLEMS DEBUG CONSOLE

/media/P/P/i/c/daoo/202402/daoo/prototipos/prototipo-20242-laravel-back

/media/P/P/i/c/daoo/202402/daoo/prototipos/prototipo-20242-laravel-back

```
> php artisan make:model Fabricante -mr --resource --api
```

INFO Model [app/Models/Fabricante.php] created successfully.

INFO Migration [database/migrations/2024_11_07_145643_create_fabricantes_table] created successfully.

INFO Controller [app/Http/Controllers/FabricanteController.php] created successfully.

Recursos e Route Model Bindings

Recursos de API



- Neste caso não foi necessário correção pois a **model** segue a **convenção de nomes** do **Laravel** (**singular/plurar** com adição de **s** do idioma **inglês**)

The screenshot shows an IDE with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, including folders like 'app', 'bootstrap', 'config', 'database', 'node_modules', 'public', 'resources', 'routes', and 'storage'. The 'routes' folder is expanded, showing 'console.php' and 'web.php'. The 'web.php' file is selected, and its content is displayed in the code editor. The code defines two routes: a resource route for 'fornecedores' using 'FornecedorController' and an API resource route for 'fabricantes' using 'FabricanteController'. The terminal at the bottom shows the command 'php artisan route:list --path=fabricante' and its output, which lists the routes for 'fabricantes' with their HTTP methods and corresponding controller actions. The output is as follows:

Method	Route	Controller Action
GET HEAD	fabricantes	fabricantes.index > FabricanteController@index
POST	fabricantes	fabricantes.store > FabricanteController@store
GET HEAD	fabricantes/{fabricante}	fabricantes.show > FabricanteController@show
PUT PATCH	fabricantes/{fabricante}	fabricantes.update > FabricanteController@update
DELETE	fabricantes/{fabricante}	fabricantes.destroy > FabricanteController@destroy

The terminal also shows the command 'php artisan route:list --path=fabricante' and the output 'Showing [5] routes'.

NOVOS VERBOS HTTP PARA A API REST

Showing [5] routes



- **REST: REpresentational State Transfer**

Método para troca de informações e serviços entre aplicações (APIs) de forma assíncrona entre uma aplicação cliente (Navegador, código **frontend**, aplicativo mobile) e uma API (**backend**) utilizando o **HTTP**:

- Conceito surgiu em 2000 como Tese de Doutorado de Roy Fielding
- Nos últimos anos tem sido mais utilizado para desenvolvimento de APIs
- Explora os verbos **HTTP**:
 - **GET**: Principalmente utilizado para **acessar/apresentar** recursos.
 - **POST**: **Criação** de **novos recursos** enviando seus dados na requisição.
 - **PUT**: **Atualização** de recursos por inteiro, **todos os dados** são atualizados.
 - **PATCH**: **Atualização** de **partes** de um recurso, alguns dados alterados.
 - **DELETE**: **Remoção** de algum recurso.



- **Códigos de Estado HTTP:**

Os códigos de resposta de *estatus* HTTP foram criados para se facilitar a verificação se a requisição foi corretamente concluída. Existem cinco classes de códigos usados para diferentes situações que podem ocorrer no processo de comunicação com o HTTP. **RFC 7231**

- Classes de códigos **HTTP** : (*Mais Detalhes* !)

- **100**: Resposta de **informação** sobre a requisição, comunicação intermediária.
- **200**: Resposta de confirmação de **sucesso** na requisição .
- **300**: Aviso de **redirecionamento** durante a requisição.
- **400**: Ocorreu algum **erro** por parte do **cliente** na geração da requisição.
- **500**: Ocorrência de **erro** por parte do **servidor** durante o processamento da resposta à requisição.

Configurando uma API Rest no Laravel

Criação de APIs REST no Laravel



- Primeiramente precisaremos instalar o módulo de API do *Laravel* com o comando abaixo:

```
$> php artisan install:api
```

- Este comando deverá gerar o arquivo ***routes/api.php***, onde configuramos as rotas da API Rest.
- O comando também pede para atualizar o banco de dados, instalando a estrutura necessária para trabalhar com autenticação via ***tokens (sanctum)***.

```
One new database migration has been published. Would you like to run all pending database migrations? (yes/no) [yes]:  
> y
```

INFO Running migrations.

```
2024_11_07_141712_create_fornecedores_table ..... 9.80ms DONE  
2024_11_07_145643_create_fabricantes_table ..... 10.07ms DONE  
2024_11_07_164056_create_personal_access_tokens_table ..... 33.28ms DONE
```

INFO API scaffolding installed. Please add the [Laravel\Sanctum\HasApiTokens] trait to your User model.

Configurando uma API Rest

Criação de APIs REST no Laravel



- Vamos criar o controlador da model **Produto** em uma pasta chamada **Api** dentro do diretório **Http/Controllers**.
- Portanto, passaremos o nome da classe com a pasta **Api/ProdutoController**. Comando do **artisan**:
- **\$> php artisan make:controller Api/ProdutoController -m Produto --resource --api**

A captura de tela mostra o ambiente de desenvolvimento. À esquerda, o 'EXPLORER' exibe a estrutura de arquivos do projeto 'PROTOTIPO-20242-LARAVEL-BACKEND-API', com a pasta 'Api' selecionada. À direita, o editor de código mostra o arquivo 'ProdutoController.php' com o seguinte conteúdo:

```
11 class ProdutoController extends Controller
22 {
23     //
29     /**
30      * Display the specified resource.
31      */
32     public function show(Produto $produto)
33     { ...
34     }
```

- Com o parâmetro **-m** já configuramos os métodos do controlador para o **Route Model Binding** e injetamos a dependência da model de **Produto**.

Configurando uma API Rest

Criação de APIs REST no Laravel



- Criaremos também as classes **Resources**, responsável pelo tratamento da resposta da API em formato **JSON**.
- Criaremos duas, **ProdutoResource** e **ProdutoCollection**

```
$> php artisan make:resource ProdutoResource
```

```
$> php artisan make:resource ProdutoCollection -c
```

- Ambos os comandos criaram os arquivos na pasta **app/Resources**:

The screenshot displays an IDE with two side-by-side code editors. The left editor shows the file explorer on the left with the 'Resources' folder expanded, containing 'ProdutoCollection.php' and 'ProdutoResource.php'. The right editor shows the code for 'ProdutoResource.php' and 'ProdutoCollection.php'. Both files start with the same imports: `use Illuminate\Http\Request;` and `use Illuminate\Http\Resources\Json\JsonResource;`. The 'ProdutoResource' class extends `JsonResource` and the 'ProdutoCollection' class extends `Resource`. Both classes have a docblock comment on line 10. The bottom panel shows the terminal with the command `php artisan make:resource ProdutoCollection -c` being executed. The terminal path is `/media/P/P/i/c/daoo/202402/daoo/prototipos/prototipo-20242-laravel-backend-api` and the prompt is `main !1 ?2`.

REST API com LARAVEL

Criação de APIs REST



- Criação do método index do ProdutoController:

- Usaremos **ProdutoCollection** no método **index** da classe **ProdutoController**.
- Passamos como parâmetro o retorno do método **all()** da classe **model** de **Produto**.
- Como resposta será retornado um JSON, com todos os produtos no atributo **data**

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController

11 class ProdutoController extends Controller
12 {
13     /**
14      * Display a listing of the resource.
15      */
16     public function index()
17     {
18         return new ProdutoCollection(Produto::all());
19     }
20     /**
    ...
    */
}
```

App\Http\Resources\ProdutoCollection

```
<?php
class ProdutoCollection extends ResourceCollection { }
```

REST API com LARAVEL

Criação de APIs REST



- **Alteração do método show do ProdutoController:**

- Usaremos **ProdutoResource** no método **index** da classe **ProdutoController**.
- Passamos como parâmetro o retorno do método **all()** da classe **model** de **Produto**.
- Como resposta será retornado um **JSON**, com os dados do produto pesquisado pelo parâmetro **ID** na url.

```
app > Http > Controllers > Api > ProdutoController.php > PHP Intelephense > ProdutoController > show

11 class ProdutoController extends Controller
28
29     /**
30      * Display the specified resource.
31      */
32     public function show(Produto $produto)
33     {
34         return new ProdutoResource($produto);
35     }
36     /**
37      *
38      */
39 
```

App\Http\Resources\ProdutoResource

```
<?php
class ProdutoResource extends JsonResource { }
```

REST API com LARAVEL

Criação de APIs REST



- **Rotas da API para o ProdutoController:**

- Para criar a rota da api para o **endpoint** de *produtos*., usaremos o método **apiResource** no arquivo **routes/api.php**.
- Desta forma geramos automaticamente as rotas para produtos e podemos testar os métodos **index** e **show** acessando as duas rotas.

The screenshot shows an IDE with the file explorer on the left displaying the project structure: `PROTOTIPO-20242-LARAVEL-BACKEND-API` with subfolders `routes`, `storage`, `tests`, and `vendor`. The `routes` folder contains `api.php`, `console.php`, and `web.php`. The main editor shows the `routes/api.php` file with the following code:

```
11  
12 Route::apiResource('produtos', ProdutoController::class);  
13
```

The bottom panel shows the terminal output of the command `php artisan route:list --path=api/produtos`:

Method	Route	Controller Action
GET HEAD	api/produtos	produtos.index > Api\ProdutoController@index
POST	api/produtos	produtos.store > Api\ProdutoController@store
GET HEAD	api/produtos/{produto}	produtos.show > Api\ProdutoController@show
PUT PATCH	api/produtos/{produto}	produtos.update > Api\ProdutoController@update
DELETE	api/produtos/{produto}	produtos.destroy > Api\ProdutoController@destroy

REST API com LARAVEL

Criação de APIs REST



- **Acessando as rotas no navegador:**
 - Para a rota **api/produtos** recebemos como resposta um objeto **JSON**, em que eu seu atributo **data** obtemos um **array** de objetos da model **Produtos**.

← → ↻ localhost:8085/api/produtos

JSON Raw Data Headers

Save Copy Collapse All Expand All Filter JSON

▼ data:

- ▶ 0: {...}
- ▼ 1:
 - id: 112
 - nome: "Notebook DELL Inspiron 15"
 - descricao: "I5 7600HQ 8GBMen GTX1030m SSD 1TB"
 - preco: 8500
 - qtd_estoque: 300
 - importado: 0
 - created_at: null
 - updated_at: null
- ▶ 2: {...}

IFSUL | CSTSI | Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br | @gillgonzales | @g1ll

REST API com LARAVEL

Criação de APIs REST



- **Acessando as rotas no navegador:**

- Para a rota **api/produtos/{ID}** recebemos como resposta um objeto **JSON**, com os dados de apenas um objeto da model **Produtos**, no atributo **data**.

← → ↻ localhost:8085/api/produtos/112

JSON Raw Data Headers

Save Copy Collapse All Expand All Filter JSON

```
▼ data:
  id: 112
  nome: "Notebook DELL Inspiron 15"
  descricao: "I5 7600HQ 8GBMen GTX1030m SSD 1TB"
  preco: 8500
  qtd_estoque: 300
  importado: 0
  created_at: null
  updated_at: null
```

Cliente HTTP

Consumindo a APIs REST com Cliente HTTP



- Para **consumir** nossas APIs de maneira mais profissional, utilizamos programas específicos como **Cientes HTTPs**, que **simulam requisições** geradas por navegadores, sem ter que desenvolver toda a parte de visualização, com *templates* e formulários.
- Existem vários **Cientes HTTP**, e dentre estes programas temos, por exemplo:
 - **Curl**: comando no terminal (`curl "protocolo://site.dominio:porta/caminho"`)
 - **Postman**: programa com um **GUI** e diversas opções de configuração, possui interface Web e necessita de criação de contas.
 - **Insomnia**: semelhante ao *postman*, necessita de conta, não possui versão com interface para a web.
- Exemplo de uma chamada a API com o comando **CURL**:

```
$> curl 'http://localhost:8085/api/produtos/112'
{"data":{"id":112,"nome":"Notebook DELL Inspiron 15","descricao":"I5 7600HQ 8GBMen GTX1030m SSD 1TB","preco":8500,"qtd_estoque":300,"importado":0,"created_at":null,"updated_at":null}}
```

Cliente HTTP

Consumindo a APIs REST com Cliente HTTP



- **POSTMAN**
- Interface com diversas funcionalidades:

The screenshot shows the Postman application interface with several annotations:

- COLEÇÕES E ORGANIZAÇÃO DAS REQUISIÇÕES**: Points to the left sidebar showing collections like 'dao0202401' and 'REST API DAO02024'.
- ABAS PARA GERENCIAR AS REQUISIÇÕES ABERTAS**: Points to the tabs at the top of the main workspace, such as 'GET INDEX' and 'GET SHOW'.
- VARIÁVEIS DE AMBIENTE**: Points to the 'envVar' button in the top right area.
- URL COM O ENDPOINT DA API**: Points to the URL input field containing 'http://localhost:8085/api/produtos'.
- AREA COM A RESPOSTA DA API**: Points to the response body area showing a JSON array of product data.

The response body shows the following JSON data:

```
1 {
2   "data": [
3     {
12    },
13    {
14      "id": 112,
15      "nome": "Notebook DELL Inspiron 15",
16      "descricao": "I5 7600HQ 8GBMen GTX1030m SSD 1TB",
17      "preco": 8500,

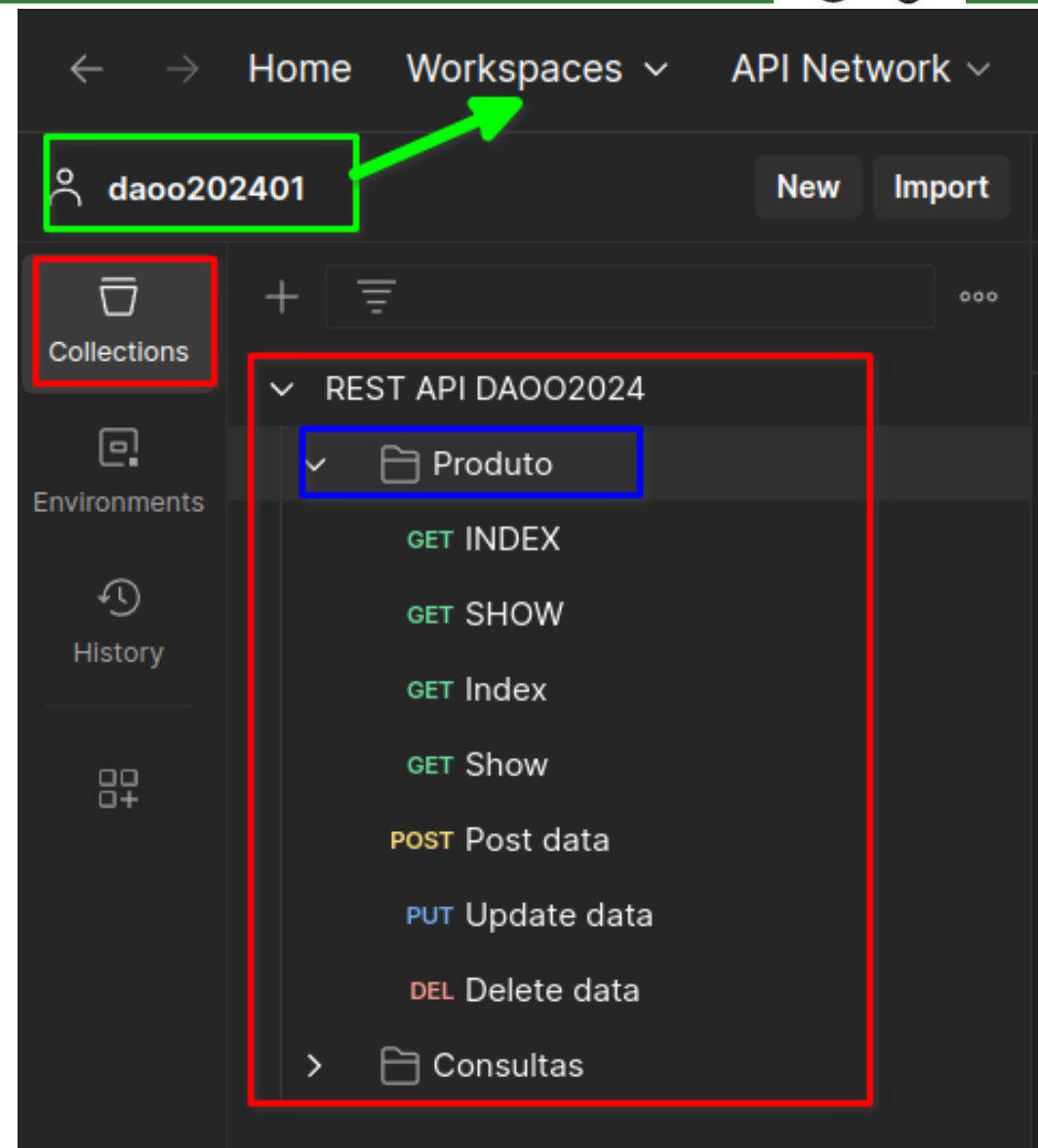
```


Cliente HTTP

Consumindo a APIs REST com Cliente HTTP



- **POSTMAN**
- O postman é organizado em espaços de trabalho (**Workspaces**) que possuem suas coleções (**Collections**)
- As coleções por sua vez possuem as requisições que criamos, que podem ser do tipo **GET**, **POST**, **PUT**, **DELETE** e demais verbos do HTTP.
- Também organizar nossas requisições em pastas, eu prefiro separar por pastas os endpoints principais, como por exemplo a **pasta Produto**, onde tenho todas as requisições para o recurso Produto.

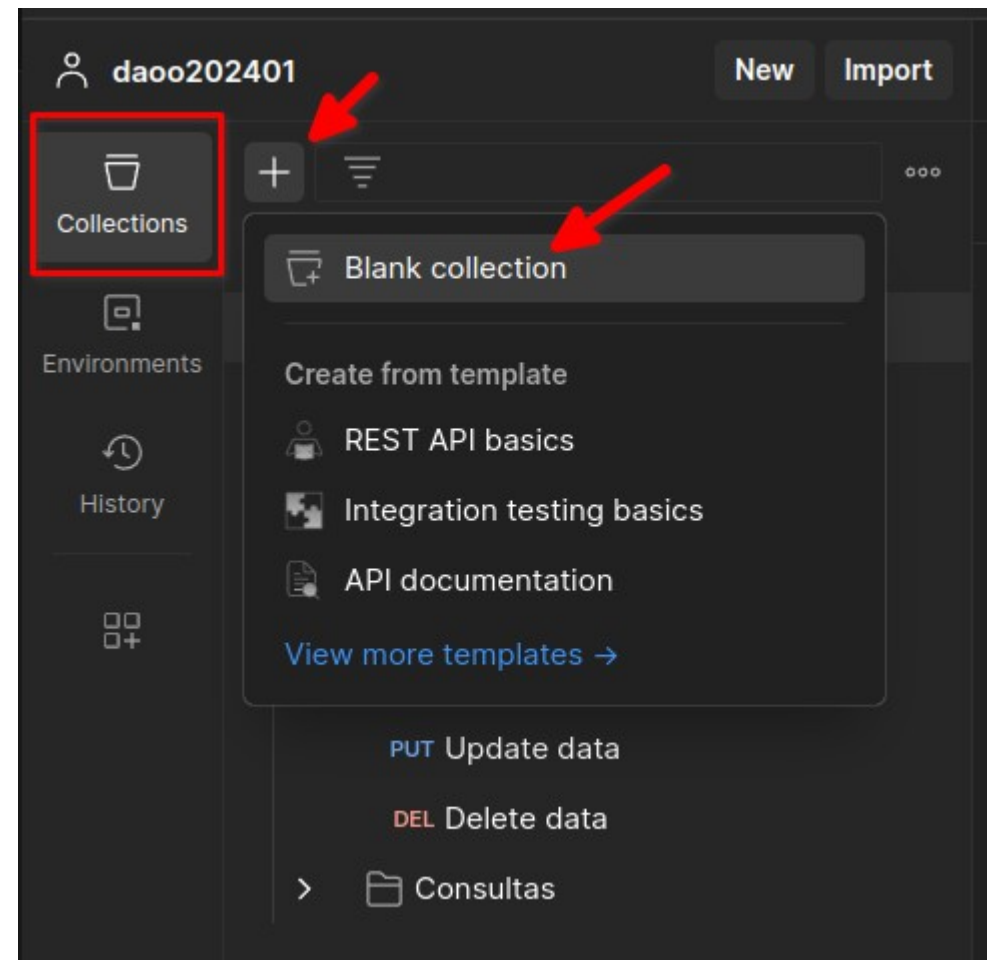
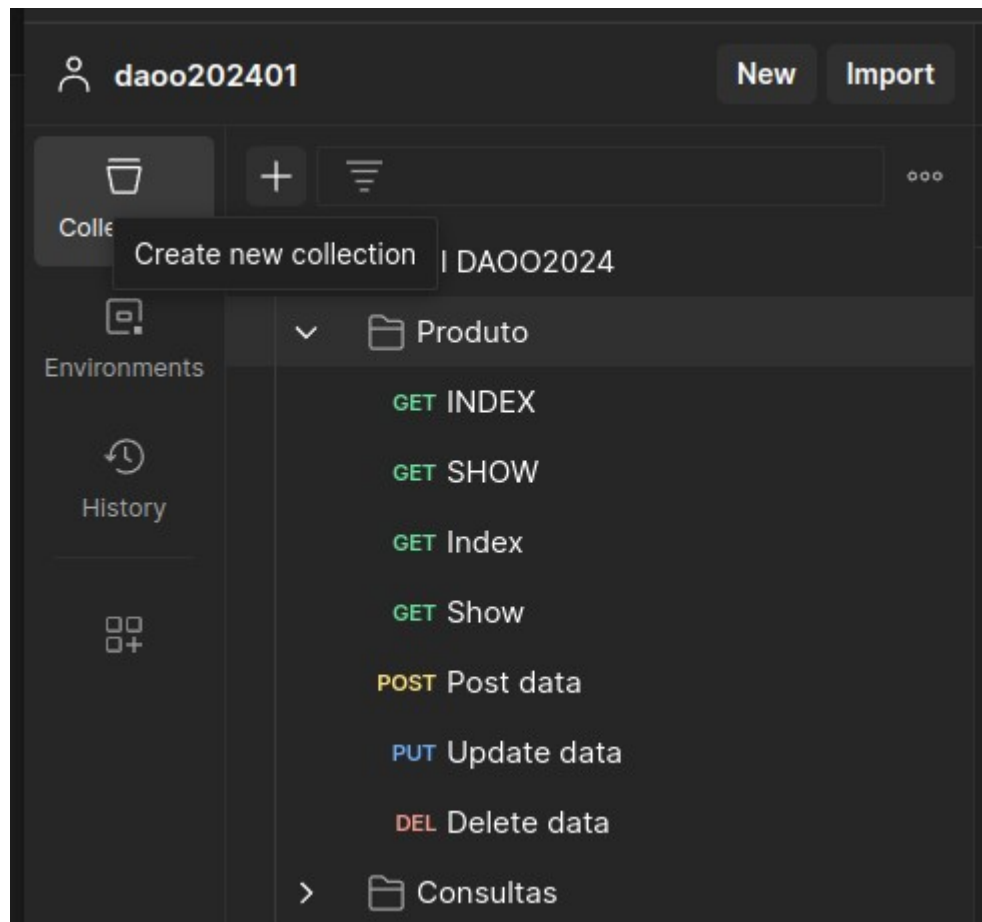


Cliente HTTP

Consumindo a APIs REST com Cliente HTTP



- Podemos ainda criar novas Coleções e agrupar requisições para uma mesma API.
- As requisições também podem ser agrupados por pastas, uma para cada recurso.



Cliente HTTP

Consumindo a APIs REST com Cliente HTTP



- Exemplo com o **Postman**: ([Vídeo de Introdução ao Postman](#) com **Vinicius Dias**)

The screenshot displays the Postman application interface. On the left sidebar, the 'collections' panel shows a collection named 'REST API DAOO2024' with a sub-collection 'Produto'. The 'environments' panel is empty. The 'History' panel shows a list of requests: 'GET INDEX', 'GET SHOW', 'GET Index', 'GET Show', 'POST Post data', 'PUT Update data', and 'DEL Delete data'. The 'Consultas' panel is also visible.

The main workspace shows a request editor for a 'GET' request to 'http://localhost:8085/api/produtos'. The request is named 'GET INDEX' and is part of the 'REST API DAOO2024' collection. The 'Params' tab is selected, showing a table with columns 'Key', 'Value', and 'Description'. The 'Body' tab is also visible, showing a JSON response.

The response viewer shows a JSON array of product data:

```
1 {
2   "data": [
3     {
12    },
13    {
14      "id": 112,
15      "nome": "Notebook DELL Inspiron 15",
16      "descricao": "I5 7600HQ 8GBMen GTX1030m SSD 1TB",
17      "preco": 8500,
18      "qtd_estoque": 300,
19      "importado": 0,
20      "created_at": null,
21      "updated_at": null
22    },
23    {
32    },
33    {

```

The status bar at the bottom right indicates 'Status: 200 OK', 'Time: 38 ms', and 'Size: 2.26 KB'. There is also a 'Save as example' button.

Cliente HTTP

Consumindo a APIs REST com Cliente HTTP



- Exemplo com o **Postman**: ([Vídeo de Introdução ao Postman](#) com **Vinicius Dias**)

The screenshot displays the Postman interface for a GET request. The URL bar shows the endpoint `http://localhost:8085/api/produtos/112`. The response status is **200 OK** with a time of 69 ms and a size of 404 B. The response body is formatted as JSON and shows the following data:

```
{
  "data": {
    "id": 112,
    "nome": "Notebook DELL Inspiron 15",
    "descricao": "I5 7600HQ 8GBMen GTX1030m SSD 1TB",
    "preco": 8500,
    "qtd_estoque": 300,
    "importado": 0,
    "created_at": null,
    "updated_at": null
  }
}
```

At the bottom of the image, there is a green banner with contact information:

IFSUL | CSTSI | Prof. Gill Velleda Gonzales | gillgonzales@ifsul.edu.br | [@gillgonzales](https://twitter.com/gillgonzales) | [@g1ll](https://twitter.com/g1ll)



1) Criar a API para três tabelas do seu projeto/TCC:

- Desenvolver os métodos para as rotas de index e show.
- Usar a configuração de rotas automáticas com apiResouces.
- Usar a injeção de dependência da Model (Route Model Binding).
- Utilizar as classes Resources para o retorno de dados em formato JSON.
- Envie o link do repositório remoto e privado na atividade referente a esta aula no moodle.
- Adicionar o professor como colaborador (**settings/colaborators**)
- Salve na pasta **database/dump** um dump do seu banco de dados com os inserts. Mantenha a camada de modelo e migrations corretamente configuradas.
- Exporte as coleções do cliente HTTP e salve na pasta **test/api** do seu projeto.



DALL'OGGIO, Pablo. **Php: programando com orientação a objetos**. 2. ed. São Paulo: Novatec, 2009

Fielding, Roy Thomas. ***Architectural Styles and the Design of Network-based Software Architectures***. Doctoral dissertation, University of California, Irvine, 2000.

Status HTTP. Mozilla Developer Network, Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>
<https://developer.mozilla.org/pt-BR/docs/Web/HTTP/Status>, acesso em: 07/11/2024

FIELDING, Roy Thomas, **Architectural Styles and the Design of Network-based Software Architectures**, Dissertation, University of California – Irvine. Disponível em: , Acesso em: 07/11/2024

<https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>

LARAVEL, **Route Model Binding**. Disponível em: <https://laravel.com/docs/11.x/routing#route-model-binding>, Acesso em: 07/11/2024.

LARAVEL, **Rest Ful Naming Resource Routes**. Disponível em: <https://laravel.com/docs/11.x/controllers#restful-naming-resource-routes>, Acesso em: 07/11/2024.

SOLID no Laravel. Disponível em: <https://dev.to/thiagoluna/solid-no-laravel-aplicando-principios-e-boas-praticas-para-entregar-melhores-solucoes-1ogh>, Acesso em: 07/11/2024.

Princípios de SOLID: O que são e como Aplicá-los no php com Laravel. Disponível em: <https://dev.to/lucascavalcante/principios-solid-o-que-sao-e-como-aplica-los-no-php-laravel-parte-01-responsabilidade-unica-3mjj>, Acesso em: 07/11/2024.

Alura – Introdução ao Postman – Disponível em: <https://www.youtube.com/watch?v=op81bMbgZXs>, Acesso em: 07/11/2024

Canal Dias de Dev – DIAS, Vinicius. Disponível em: <https://www.youtube.com/@DiasDeDev/videos>, Acesso em: 07/11/2024.